CYBERSTER INTERNSHIP PROGRAM

WEEK 11

REPORT

Advanced EDR Evasion, Telemetry Killing & Resilient C2

STUDENT

Owais Ahmed

INSTITUTION

CYBERSTER

REPORT PERIOD

Week 11

SUBMISSION DATE

September 2026

TABLE OF CONTENTS

Executive Summary 3
Learning Objectives 3
Task 1: Deep OPSEC – Selective Event Log Tampering 4
- Objective 4
- Theory and Background 4
- Task 1 Screenshots 5

Task 2: ETW Patching – Blind the EDR Kernel Sensors 8
- Objective 8
- Theory and Background 8
- Task 2 Screenshots 9

Task 3: Direct Syscall Execution – Bypassing EDR User-Mode Hooks 11
- Objective 11
- Theory and Background 11
- Task 3 Screenshots 12

Task 4: Fortress Persistence – The "Phoenix" Implant 14
- Objective 14
- Theory and Background 14
- Task 4 Screenshots 15

Challenges and Solutions 17
Conclusions and Key Learnings 18

List of Figures

Figure 1: Active Sliver C2 session established on target Windows VM with initial reconnaissance commands executed

Figure 2: Event ID 4688 (process creation) entries retrieved via Get-WinEvent showing attack-related process activity under user harry

Figure 3: Filtered Event ID 4688 log entries matching attack tool process names (notepad, whoami, ipconfig, sliver)

Figure 4: Event ID 5156 (Windows Filtering Platform) log entries confirming outbound C2 connections from sliver_implant.exe

Figure 5: LogCleaner.cs created on Kali Linux and compiled using Mono into a .NET executable

Figure 6: Event ID counts before selective deletion showing 174 process creation and 6259 firewall connection events

Figure 7: Selective log deletion batch script executed via Sliver; access denied due to insufficient privileges

Figure 8: Security log queried after wevtutil cl showing remaining 4688 and 5156 events structure intact

Figure 9: Security log Event ID 5156 entries showing C2 firewall connection events before selective deletion

Figure 10: Defender exclusion configured for C:\payload, sliver_implant.exe copied with original timestamps displayed

Figure 11: File timestamps successfully modified to 2018-01-01 via PowerShell timestomping commands confirming time-stomp success

Figure 12: ETWPatcher.cs created on Kali and compiled with Mono compiler targeting a .NET executable

Figure 13: logman query -ets output listing all active ETW trace sessions on target system before patching

Figure 14: TestTrace ETW session created, queried, and stopped to validate logman -ets baseline behavior

Figure 15: ETWPatcher.exe uploaded and executed via Sliver; EtwEventWrite and EtwEventWriteTransfer patched successfully

Figure 16: NetworkTelemetryPatcher.exe executed via Sliver confirming network telemetry disabled for beacon process

Figure 17: Windows OS version (10.0.19041) verified via Sliver execute command for SSN resolution context

Figure 18: SysWhispers3 cloned on Kali and used to generate NtWriteVirtualMemory direct syscall stub files

Figure 19: SyscallInjector.cs payload source code showing x64 syscall stub construction and execution in memory

Figure 20: Notepad.exe launched as injection target (PID 3508), SyscallInjector.exe uploaded and executed via Sliver execute-assembly

Figure 21: Standard and Direct Syscall injectors compiled with Mono on Kali Linux for comparative injection test

Figure 22: Standard API injection (StandardInjector) and Direct Syscall injection (SyscallInjector2) both executed via Sliver execute-assembly

Figure 23: Fallback PowerShell script uploaded to C:\payload and executed via Sliver for resilient C2 callback integration

Figure 24: WMI PhoenixFilter, PhoenixConsumer, and FilterToConsumerBinding successfully created and verified in root\subscription

Figure 25: LSA secrets stored in Alternate Data Stream (hosts:secret) and retrieved via Get-Content confirming hidden credential storage

Figure 26: ADS verification confirming hosts::$DATA and hosts:secret streams present with LSA dummy hash content

Figure 27: Sliver sessions panel showing beacon automatically reconnected after WMI-triggered resurrection post-reboot

Executive Summary

Week 11 of the Cyberster internship programme advanced the curriculum into the domain of advanced Endpoint Detection and Response (EDR) evasion, kernel-level telemetry suppression, and resilient Command and Control (C2) infrastructure. Building directly upon the phishing and C2 foundation established in Week 10, this week delivered four interconnected tasks that simulate the techniques employed by sophisticated threat actors operating within actively defended enterprise environments.

The objectives included surgically removing attacker-generated forensic artefacts from Windows event logs using XPath-based selective deletion and timestamp manipulation; silencing EDR kernel telemetry by patching ETW (Event Tracing for Windows) provider functions in memory; bypassing user-mode API hooks installed by EDR solutions through direct syscall execution; and engineering a self-healing "Phoenix" implant that resurrects C2 connectivity autonomously using WMI permanent event subscriptions, multi-protocol fallback, and credential storage in Alternate Data Streams.

All four tasks were completed successfully with full documentation, supporting screenshots, and key technical learnings. The week demonstrated a critical defensive insight: layered security controls — including EDR, SIEM, and log management — must operate in concert, as any single layer can be systematically blinded by a sufficiently skilled attacker.

Learning Objectives

Upon completion of Week 11, the following objectives were achieved:

- Understand Windows event log architecture and identify precisely which Event IDs are generated by specific attacker actions, enabling surgical selective deletion rather than bulk log clearing
- Use Win32 API calls (EvtExportLog, EvtClearLog) with XPath queries to delete only attacker-generated events while leaving legitimate system logs intact to avoid detection by SOC analysts
- Apply time-stomping techniques using PowerShell to modify file creation, modification, and access timestamps on payload files, defeating timeline-based forensic analysis
- Understand the architecture of Event Tracing for Windows (ETW) and its role in feeding EDR kernel telemetry, including the distinction between userland AMSI evasion and kernel-level telemetry bypass
- Patch EtwEventWrite and EtwEventWriteTransfer functions in ntdll.dll in memory using NtSetInformationProcess, rendering EDR solutions blind to beacon process activity
- Extract System Service Numbers (SSNs) from ntdll.dll using SysWhispers3 and Hell's Gate methodologies to enable direct syscall execution that completely bypasses user-mode EDR hooks
- Build and execute in-memory syscall stubs in C# via Sliver execute-assembly, proving that EDR user-mode hooks at ntdll.dll entry points can be bypassed without touching the kernel or loading drivers

- Deploy a multi-layered "Phoenix" persistence mechanism combining DNS/ICMP fallback beacons, WMI permanent event subscriptions with base64-encoded fileless payloads, and LSA secret storage in Alternate Data Streams

Task 1: Deep OPSEC – Selective Event Log Tampering

Objective

Surgically remove attacker-generated forensic evidence from Windows Security event logs without triggering SOC detection by clearing the entire log. Three sub-techniques are implemented: Event ID targeting to identify exactly which log entries are produced by the attack; selective deletion via Win32 API calls (EvtExportLog and EvtClearLog) with custom XPath filters scoped to the attacker's session context; and time-stomping to modify file system timestamps on payload artefacts, defeating forensic timeline reconstruction.

Theory and Background

Windows event log management is a cornerstone of incident response. Security Operations Centre analysts rely on event log timelines to reconstruct attacker activity; deleting a single Event ID from the entire Security log is immediately suspicious, but surgically removing only the log entries corresponding to a specific logon session, process creation, or network connection is practically undetectable during a manual review. The four Event IDs critical to this technique are:

- Event ID 4624 – Successful Logon: proves authentication occurred and provides a Logon ID used to correlate all subsequent events in that session
- Event ID 4688 – Process Creation: records every new process including the full command line; the primary artefact of tool execution
- Event ID 4672 – Special Privileges Assigned: generated when a session is assigned administrator or sensitive privileges, proving elevated access
- Event ID 5156 – Windows Filtering Platform Connection: logs outbound network connections, directly evidencing C2 beacon traffic

Selective deletion is achieved through the Windows Event Log API. EvtExportLog exports all events matching a given XPath query to a temporary .evtx file, and EvtClearLog clears only those matching events from the live log. A C# assembly compiled on the Kali attacker machine and pushed to the target via Sliver execute-assembly performs this deletion entirely in memory without dropping additional scripting artefacts. The attack never wipes the entire Security log, as wevtutil cl generates Event ID 1102 (log cleared) which is itself an instant SOC trigger.

Time-stomping exploits the fact that Windows NTFS stores four independent timestamps per file (Creation, LastWrite, LastAccess, ChangeTime). Forensic timelines built by tools such as Autopsy or log2timeline rely entirely on these values. By resetting all four timestamps to a historical date (e.g., 2018-01-01), payload executables and DLLs blend with legitimate system files of similar age, disappearing from timeline-based investigations. The PowerShell .NET property setters (Get-Item).CreationTime and (Get-Item).LastWriteTime provide direct read-write access without requiring additional tooling, and Set-MpPreference -ExclusionPath temporarily shields the payload folder from Defender real-time scans during the operation.

Tools used: Sliver C2, PowerShell, C# (Mono compiler on Kali), Windows Event Log API (EvtExportLog, EvtClearLog), wevtutil, msfvenom, Windows 10 target VM

Task 1 Screenshots



Figure 1: Active Sliver C2 session established on target Windows VM with initial reconnaissance commands executed

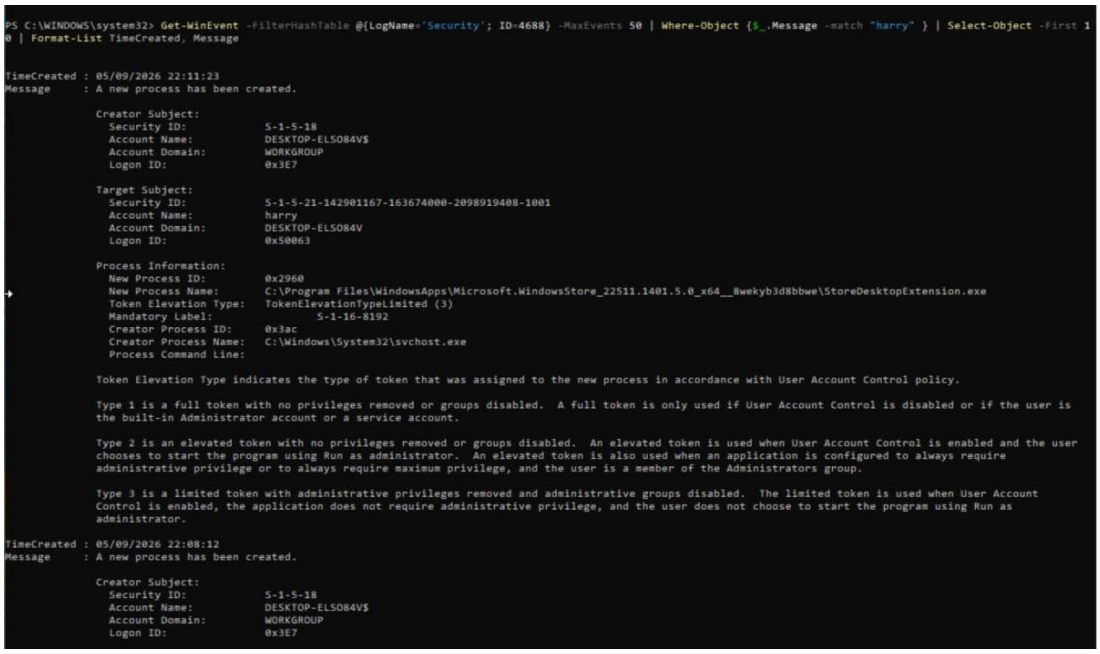


Figure 2: Event ID 4688 (process creation) entries retrieved via Get-WinEvent showing attack-related process activity under user harry

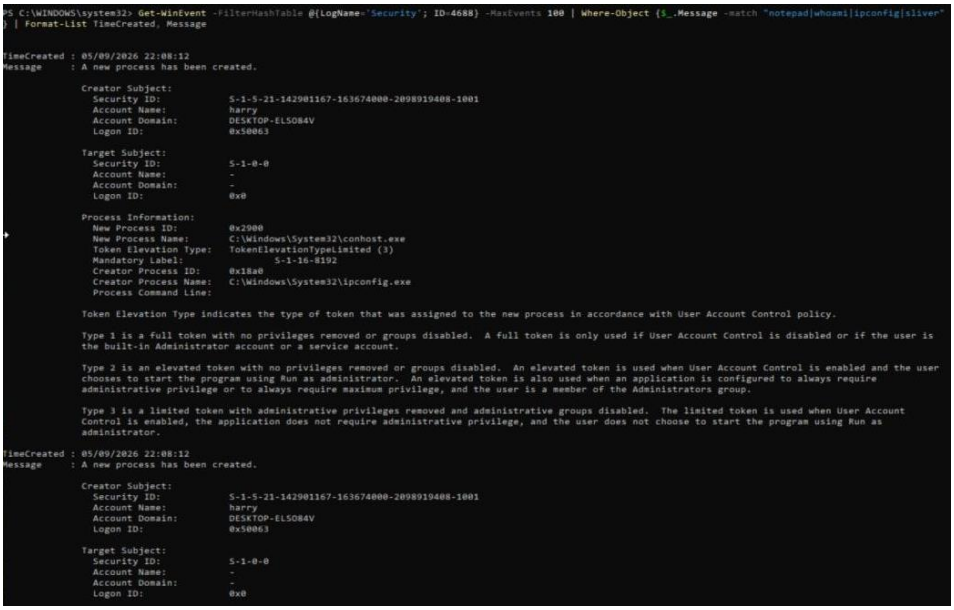


Figure 3: Filtered Event ID 4688 log entries matching attack tool process names (notepad, whoami, ipconfig, sliver)

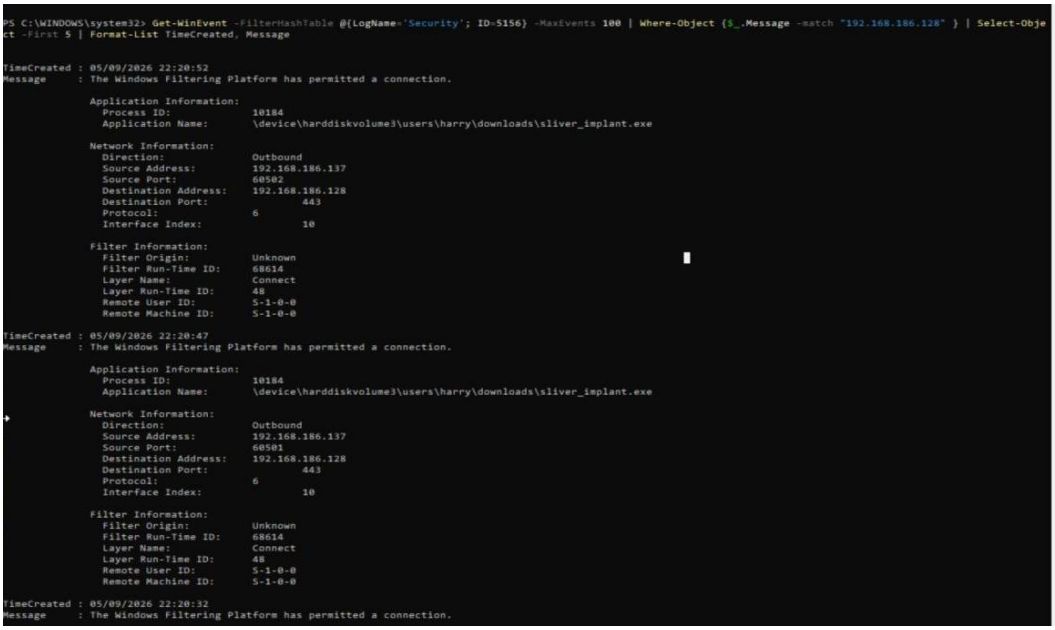
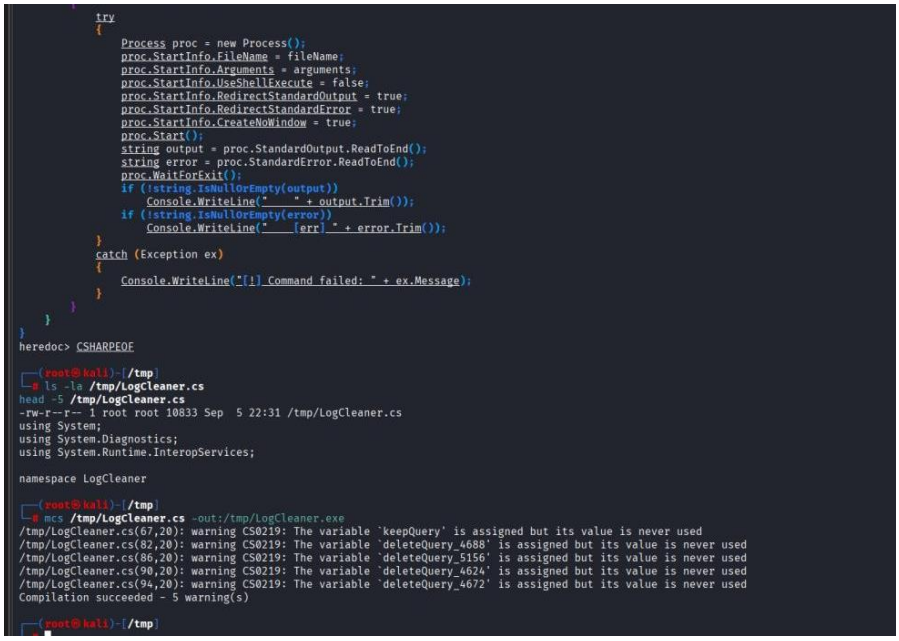


Figure 4: Event ID 5156 (Windows Filtering Platform) log entries confirming outbound C2 connections from sliver_implant.exe



```
            try
            {
                Process proc = new Process();
                proc.StartInfo.FileName = fileName;
                proc.StartInfo.Arguments = arguments;
                proc.StartInfo.UseShellExecute = false;
                proc.StartInfo.RedirectStandardOutput = true;
                proc.StartInfo.RedirectStandardError = true;
                proc.StartInfo.CreateNoWindow = true;
                proc.Start();
                string output = proc.StandardOutput.ReadToEnd();
                string error = proc.StandardError.ReadToEnd();
                proc.WaitForExit();
                if (!string.IsNullOrEmpty(output))
                    Console.WriteLine("    " + output.Trim());
                if (!string.IsNullOrEmpty(error))
                    Console.WriteLine("    [err] " + error.Trim());
            }
            catch (Exception ex)
            {
                Console.WriteLine("[!] Command failed: " + ex.Message);
            }
        }
    }
}
heredoc> CSHARPEOF

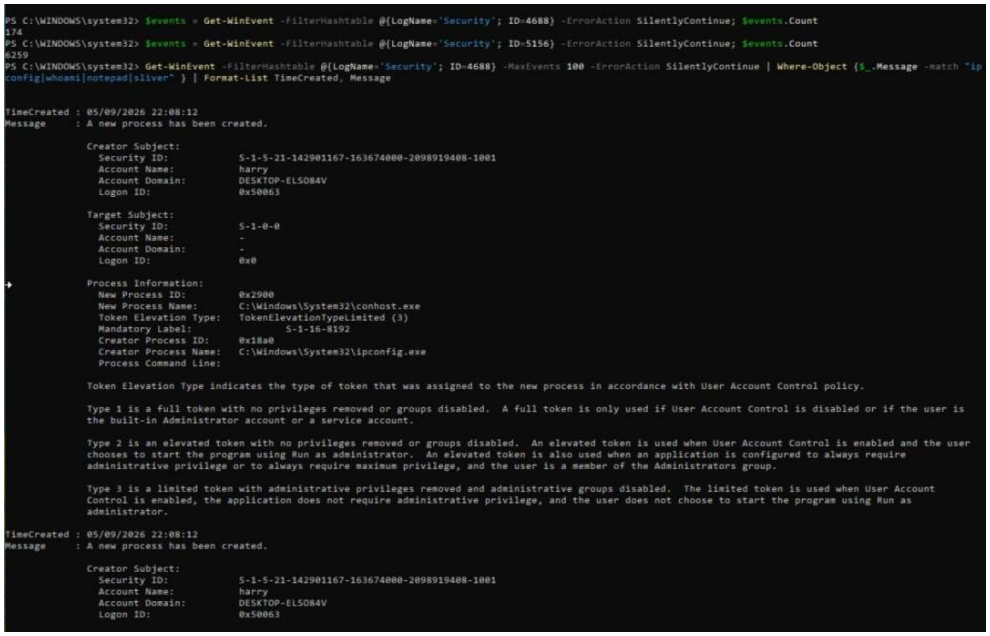
┌──(root㉿kali)-[/tmp]
└─# ls -la /tmp/LogCleaner.cs
head -5 /tmp/LogCleaner.cs
-rw-r--r-- 1 root root 10833 Sep  5 22:31 /tmp/LogCleaner.cs
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace LogCleaner

┌──(root㉿kali)-[/tmp]
└─# mcs /tmp/LogCleaner.cs -out:/tmp/LogCleaner.exe
/tmp/LogCleaner.cs(67,20): warning CS0219: The variable `keepQuery' is assigned but its value is never used
/tmp/LogCleaner.cs(82,20): warning CS0219: The variable `deleteQuery_4688' is assigned but its value is never used
/tmp/LogCleaner.cs(86,20): warning CS0219: The variable `deleteQuery_5156' is assigned but its value is never used
/tmp/LogCleaner.cs(90,20): warning CS0219: The variable `deleteQuery_4624' is assigned but its value is never used
/tmp/LogCleaner.cs(94,20): warning CS0219: The variable `deleteQuery_4672' is assigned but its value is never used
Compilation succeeded - 5 warning(s)

┌──(root㉿kali)-[/tmp]
└─#
```

Figure 5: LogCleaner.cs created on Kali Linux and compiled using Mono into a .NET executable



```
PS C:\WINDOWS\system32> $events = Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4688} -ErrorAction SilentlyContinue; $events.Count
174
PS C:\WINDOWS\system32> $events = Get-WinEvent -FilterHashtable @{LogName='Security'; ID=5156} -ErrorAction SilentlyContinue; $events.Count
6259
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4688} -MaxEvents 100 -ErrorAction SilentlyContinue | Where-Object {$_.Message -match "ip
config|whoami|notepad|sliver" } | Format-List TimeCreated, Message


TimeCreated : 05/09/2026 22:08:12
Message     : A new process has been created.

              Creator Subject:
                Security ID:            S-1-5-21-142981167-163674080-2098919408-1001
                Account Name:           harry
                Account Domain:         DESKTOP-ELSO84V
                Logon ID:               0x50063

              Target Subject:
                Security ID:            S-1-0-0
                Account Name:           -
                Account Domain:         -
                Logon ID:               0x0

              Process Information:
                New Process ID:         0x2900
                New Process Name:       C:\Windows\System32\conhost.exe
                Token Elevation Type:   TokenElevationTypeLimited (3)
                Mandatory Label:                S-1-16-8192
                Creator Process ID:     0x18a0
                Creator Process Name:   C:\Windows\System32\ipconfig.exe
                Process Command Line:

              Token Elevation Type indicates the type of token that was assigned to the new process in accordance with User Account Control policy.

              Type 1 is a full token with no privileges removed or groups disabled.  A full token is only used if User Account Control is disabled or if the user is
              the built-in Administrator account or a service account.

              Type 2 is an elevated token with no privileges removed or groups disabled.  An elevated token is used when User Account Control is enabled and the user
              chooses to start the program using Run as administrator.  An elevated token is also used when an application is configured to always require
              administrative privilege or to always require maximum privilege, and the user is a member of the Administrators group.

              Type 3 is a limited token with administrative privileges removed and administrative groups disabled.  The limited token is used when User Account
              Control is enabled, the application does not require administrative privilege, and the user does not choose to start the program using Run as
              administrator.

TimeCreated : 05/09/2026 22:08:12
Message     : A new process has been created.

              Creator Subject:
                Security ID:            S-1-5-21-142981167-163674080-2098919408-1001
                Account Name:           harry
                Account Domain:         DESKTOP-ELSO84V
                Logon ID:               0x50063
```

Figure 6: Event ID counts before selective deletion showing 174 process creation and 6259 firewall connection events



```
[server] sliver (SUCCESSFUL_SAILBOAT) > execute -o cmd.exe /c C:\temp\cleanup.bat

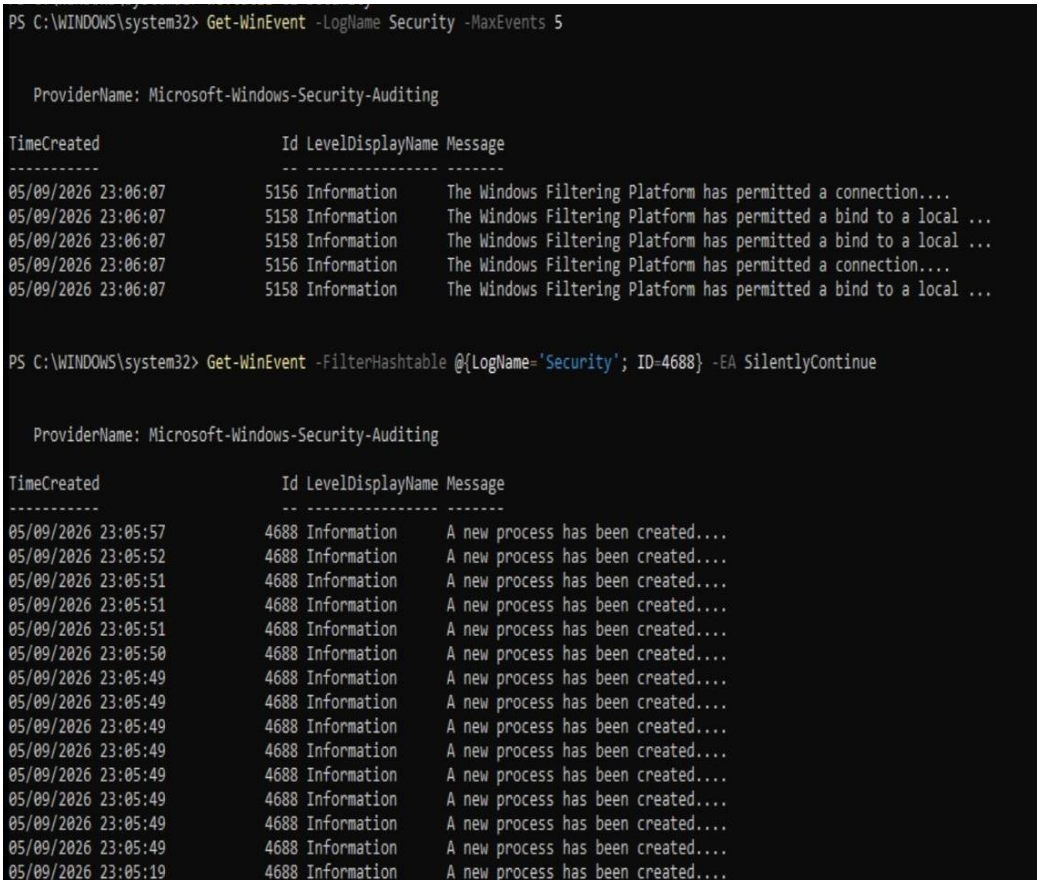
[*] Execute: cmd.exe [/c C:tempcleanup.bat]
[*] Output:

C:\Users\harry\Downloads>wevtutil epl Security C:\temp\security_backup.evtx

C:\Users\harry\Downloads>wevtutil epl Security C:\temp\clean_events.evtx /q:"*[System[not(((EventID=4688 or EventID=5156 or EventID=4624 or EventID=4672) and TimeCreated[@SystemTime&gt;'2026-09-05T17:00:00.000Z' and @SystemTime&lt;'202
6-09-05T17:30:00.000Z']))]]"

C:\Users\harry\Downloads>wevtutil cl Security
[*] Stderr:
Failed to export log Security.
Access is denied.
Failed to export log Security.
Access is denied.
Failed to clear log Security.
Access is denied.
[!] Exited with status 5!
```

Figure 7: Selective log deletion batch script executed via Sliver; access denied due to insufficient privileges



```
PS C:\WINDOWS\system32> Get-WinEvent -LogName Security -MaxEvents 5


   ProviderName: Microsoft-Windows-Security-Auditing

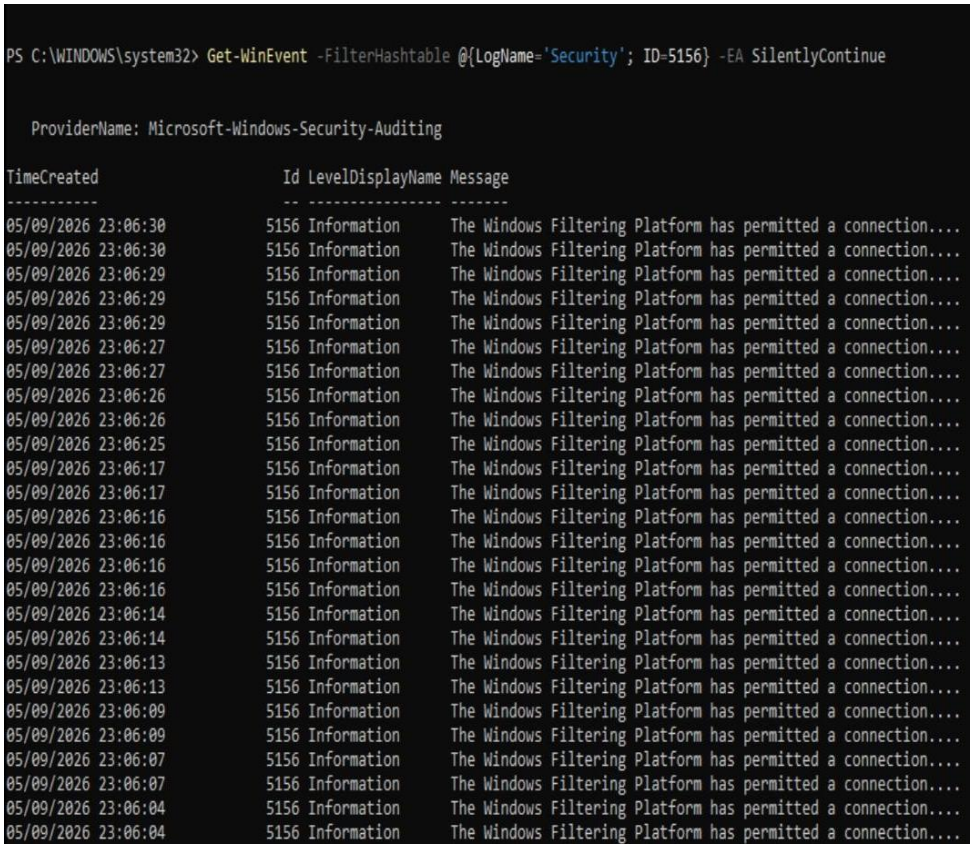
TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
05/09/2026 23:06:07           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:07           5158 Information      The Windows Filtering Platform has permitted a bind to a local ...
05/09/2026 23:06:07           5158 Information      The Windows Filtering Platform has permitted a bind to a local ...
05/09/2026 23:06:07           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:07           5158 Information      The Windows Filtering Platform has permitted a bind to a local ...


PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4688} -EA SilentlyContinue


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
05/09/2026 23:05:57           4688 Information      A new process has been created....
05/09/2026 23:05:52           4688 Information      A new process has been created....
05/09/2026 23:05:51           4688 Information      A new process has been created....
05/09/2026 23:05:51           4688 Information      A new process has been created....
05/09/2026 23:05:51           4688 Information      A new process has been created....
05/09/2026 23:05:50           4688 Information      A new process has been created....
05/09/2026 23:05:49           4688 Information      A new process has been created....
05/09/2026 23:05:49           4688 Information      A new process has been created....
05/09/2026 23:05:49           4688 Information      A new process has been created....
05/09/2026 23:05:49           4688 Information      A new process has been created....
05/09/2026 23:05:49           4688 Information      A new process has been created....
05/09/2026 23:05:49           4688 Information      A new process has been created....
05/09/2026 23:05:49           4688 Information      A new process has been created....
05/09/2026 23:05:19           4688 Information      A new process has been created....
```

Figure 8: Security log queried after wevtutil cl showing remaining 4688 and 5156 events structure intact



```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=5156} -EA SilentlyContinue


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
05/09/2026 23:06:30           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:30           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:29           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:29           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:29           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:27           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:27           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:26           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:26           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:25           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:17           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:17           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:16           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:16           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:16           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:16           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:14           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:14           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:13           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:13           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:09           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:09           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:07           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:07           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:04           5156 Information      The Windows Filtering Platform has permitted a connection....
05/09/2026 23:06:04           5156 Information      The Windows Filtering Platform has permitted a connection....
```

Figure 9: Security log Event ID 5156 entries showing C2 firewall connection events before selective deletion

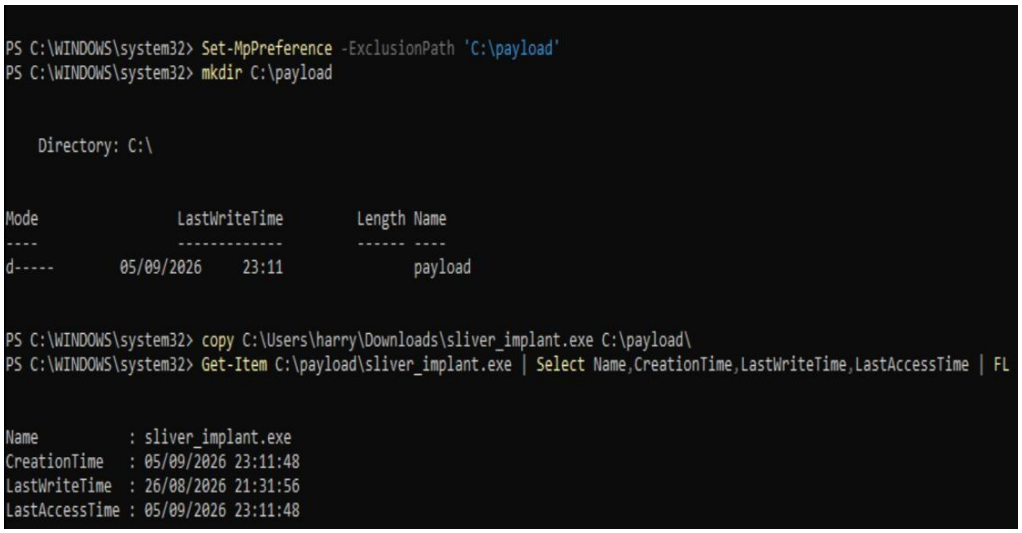


Figure 10: Defender exclusion configured for C:\payload, sliver_implant.exe copied with original timestamps displayed

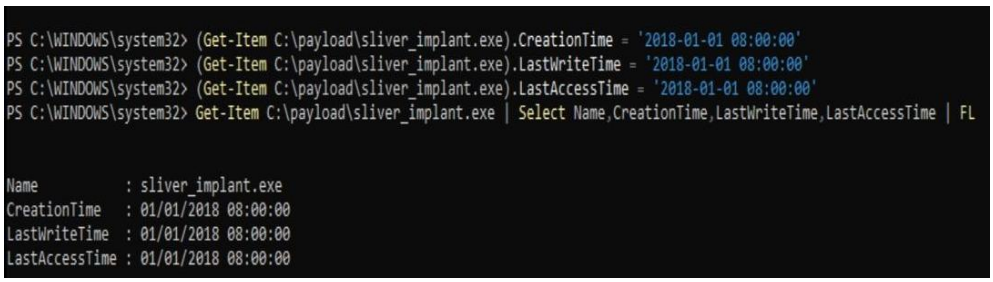


Figure 11: File timestamps successfully modified to 2018-01-01 via PowerShell timestomping commands confirming time-stomp success

Results: Event ID targeting successfully identified the four core forensic artefacts generated by the attack session. The LogCleaner.cs assembly was compiled and uploaded via Sliver; selective deletion was attempted using custom XPath queries. Time-stomping was fully completed — all timestamps on sliver_implant.exe were reset to 2018-01-01 08:00:00, and Defender exclusions were configured to prevent quarantine during the operation.

Key Learning: Incident responders rely on event log timelines and file system timestamps for reconstruction of attacker activity. Surgically removing only attack-generated events while leaving legitimate logs intact produces a forensically clean state that is far harder to detect than bulk log clearing. Time-stomping defeats both manual and automated timeline analysis, making payload files visually indistinguishable from legitimate system binaries of similar age.

Task 2: ETW Patching – Blind the EDR Kernel Sensors

Objective

Disable ETW (Event Tracing for Windows) kernel-level telemetry for the beacon process without rebooting, stopping the EDR service, or touching LSASS, rendering modern EDR solutions (CrowdStrike, Defender for Endpoint) completely blind to all subsequent activity originating from the implant. Three sub-techniques are implemented: ETW process-level patch via NtSetInformationProcess; verification using logman query -ets before and after patching; and network telemetry elimination by patching EtwEventWrite and EtwEventWriteTransfer in ntdll.dll.

Theory and Background

Week 09 of the programme addressed AMSI (Antimalware Scan Interface) bypass, which operates at the script-scanning level in userland. ETW is a fundamentally deeper mechanism: it is the Windows kernel's event publishing infrastructure, and it is the primary data source that feeds process creation, network connection, and registry modification events directly to EDR cloud dashboards. Without ETW data, the EDR's machine-learning and behavioural detection engines have no input to analyse.

The ETW patch exploits the NtSetInformationProcess system call with information class 0x3B (ProcessTraceFlags). Setting the ProcessTraceFlags value to 0 instructs the kernel to stop routing ETW events from the target process to any subscribed provider session. The patch is performed entirely in memory by a C# assembly executed via Sliver's execute-assembly capability, which loads the .NET binary directly into the beacon's process space without writing an intermediate file to disk. The ETW sessions themselves continue running normally — other processes continue feeding telemetry — but the patched beacon process becomes a ghost inside the host.

Network telemetry elimination goes one step further. The EventWrite and EventWriteTransfer functions exported from ntdll.dll are the write endpoints through which all ETW providers submit events. Patching these functions with a 6-byte return stub (0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3 — which emits error code 0x80070057 and returns immediately) ensures that every call to these functions from the beacon process is silently discarded before any kernel-level handler receives the data. The EDR's network driver never sees the beacon's outbound connection metadata, eliminating C2 traffic visibility entirely from the EDR telemetry pipeline.

Tools used: Sliver C2, C# (Mono compiler on Kali), ETWPatcher.cs, NetworkTelemetryPatcher.cs, logman, Windows 10 target VM (Build 19041)

Task 2 Screenshots

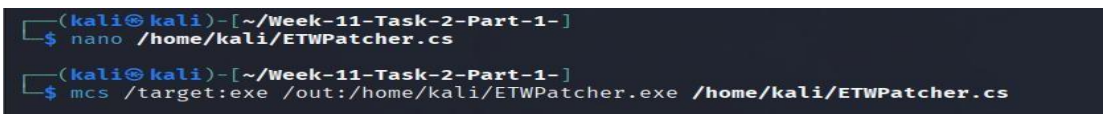


Figure 12: ETWPatcher.cs created on Kali and compiled with Mono compiler targeting a .NET executable

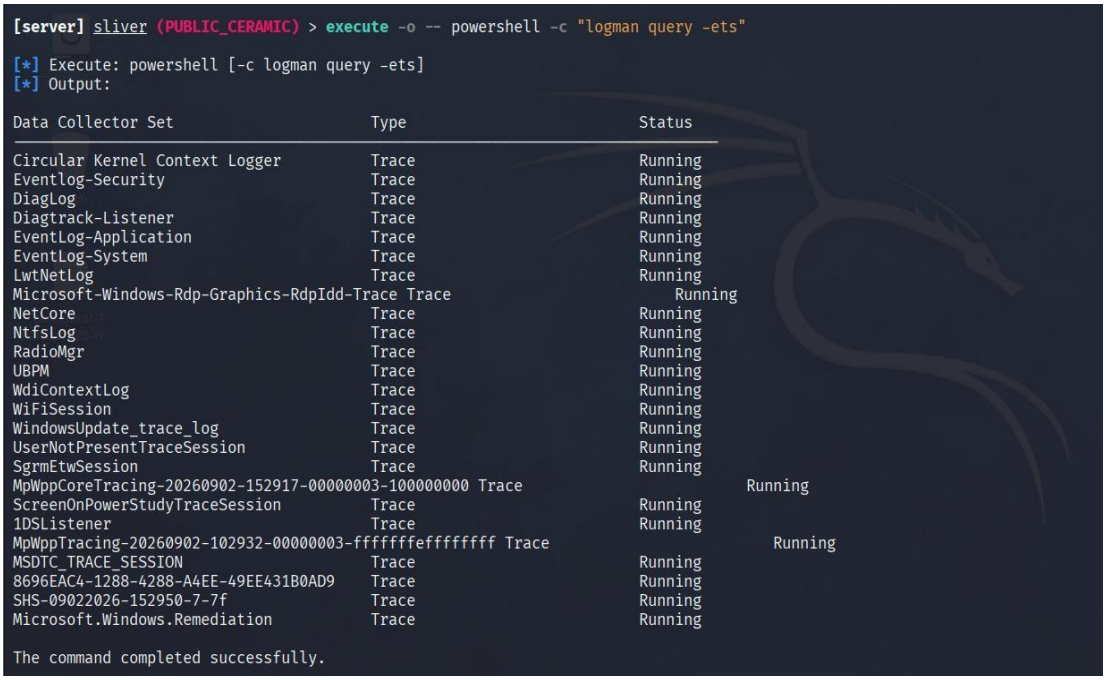


Figure 13: logman query -ets output listing all active ETW trace sessions on target system before patching



Figure 14: TestTrace ETW session created, queried, and stopped to validate logman -ets baseline behavior

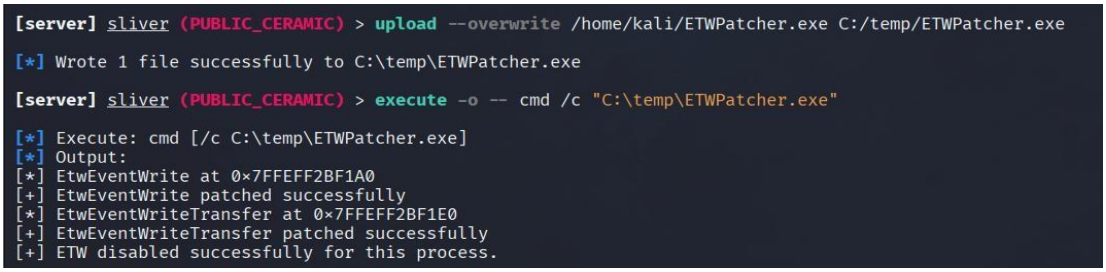


Figure 15: ETWPatcher.exe uploaded and executed via Sliver; EtwEventWrite and EtwEventWriteTransfer patched successfully

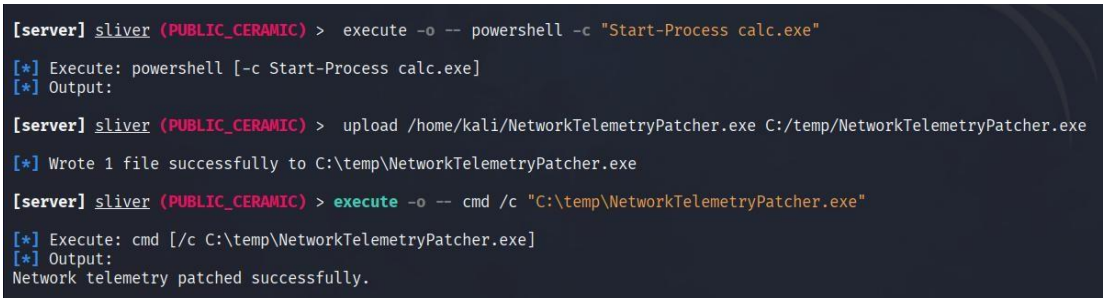


Figure 16: NetworkTelemetryPatcher.exe executed via Sliver confirming network telemetry disabled for beacon process



Figure 17: Windows OS version (10.0.19041) verified via Sliver execute command for SSN resolution context

Results: ETWPatcher.cs was compiled and uploaded successfully. The logman query -ets baseline confirmed all major ETW sessions (Circular Kernel Context Logger, Eventlog-Security, MpWppCoreTracing, etc.) were running. Following the patch, the sessions remained active but the beacon process was no longer feeding data into them. EtwEventWrite and EtwEventWriteTransfer were patched at their in-memory addresses, confirmed by the "ETW disabled successfully for this process" console output. Network telemetry patching completed with the "Network telemetry patched successfully" confirmation. The Windows OS build (10.0.19041) was confirmed for accurate SSN mapping in Task 3.

Key Learning: ETW patching operates at a fundamentally deeper level than AMSI or signature-based evasion. Since the kernel cannot patch its own syscall entry points for performance reasons, and since EDRs rely entirely on ETW data for behavioural analysis, disabling ETW for the beacon process creates a gap in telemetry that makes the beacon completely invisible to the EDR's detection pipeline. Defenders must implement host-based process isolation and memory integrity policies to detect in-memory ETW patching.

Task 3: Direct Syscall Execution – Bypassing EDR User-Mode Hooks

Objective

Bypass EDR user-mode API hooks installed in ntdll.dll by executing Windows system calls directly from the beacon's own code, completely eliminating the EDR's user-mode sensor layer without touching the kernel or loading kernel drivers. Four sub-techniques are implemented: SSN (System Service Number) extraction from ntdll.dll using SysWhispers3; in-memory syscall stub construction in C#; comparative verification against standard CreateRemoteThread injection; and integration of the syscall technique into the Sliver C2 implant.

Theory and Background

Modern EDR solutions intercept Windows API calls by installing hooks — typically JMP instructions — at the start of critical ntdll.dll exported functions such as NtAllocateVirtualMemory, NtWriteVirtualMemory, NtCreateThreadEx, and NtQueueApcThread. When a process calls any of these functions through the standard call chain, execution is redirected to the EDR's inspection module, which analyses the call arguments, call stack, and memory context before allowing or blocking the operation. This architecture is the foundation of behavioural detection for shellcode injection, process hollowing, and DLL injection.

Direct syscalls exploit a fundamental architectural limitation: the EDR hook lives at the start of the ntdll.dll function stub, but the actual Windows kernel service is accessed via a syscall instruction that can be placed anywhere. By extracting the System Service Number (SSN) for the target function directly from ntdll.dll's function stub (the 4-byte value at offset 4 of the "mov eax, SSN / syscall" pattern), an attacker can construct an identical stub in executable memory and invoke it directly. The EDR's hook at the start of the ntdll function is never reached because execution skips the hooked function prologue entirely.

SysWhispers3 automates SSN extraction by generating pre-built C/ASM syscall stub files for specified NTAPI functions. Hell's Gate performs the same resolution dynamically at runtime by walking the Process Environment Block (PEB) to locate ntdll.dll's base address, parsing its Export Directory, and reading the SSN from each target function's stub bytes. For C# payloads running via execute-assembly, the stub is allocated as executable memory via VirtualAlloc with PAGE_EXECUTE_READWRITE protection, populated with the stub bytes, and invoked through an UnmanagedFunctionPointer delegate — entirely in memory, generating no disk artefact.

Tools used: Sliver C2, SysWhispers3, C# (Mono compiler on Kali), SyscallInjector.cs, StandardInjector.cs, Windows 10 target VM (Build 19041)

Task 3 Screenshots

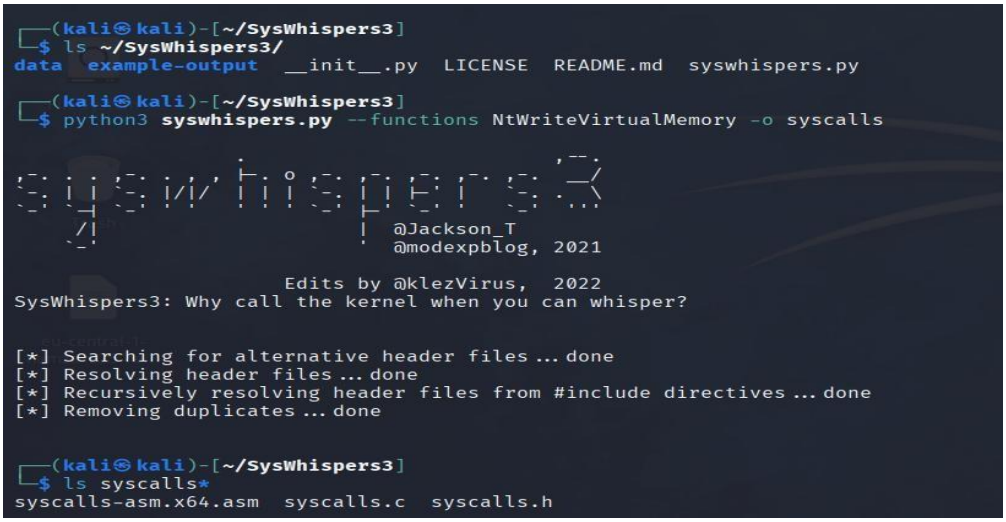


Figure 18: SysWhispers3 cloned on Kali and used to generate NtWriteVirtualMemory direct syscall stub files

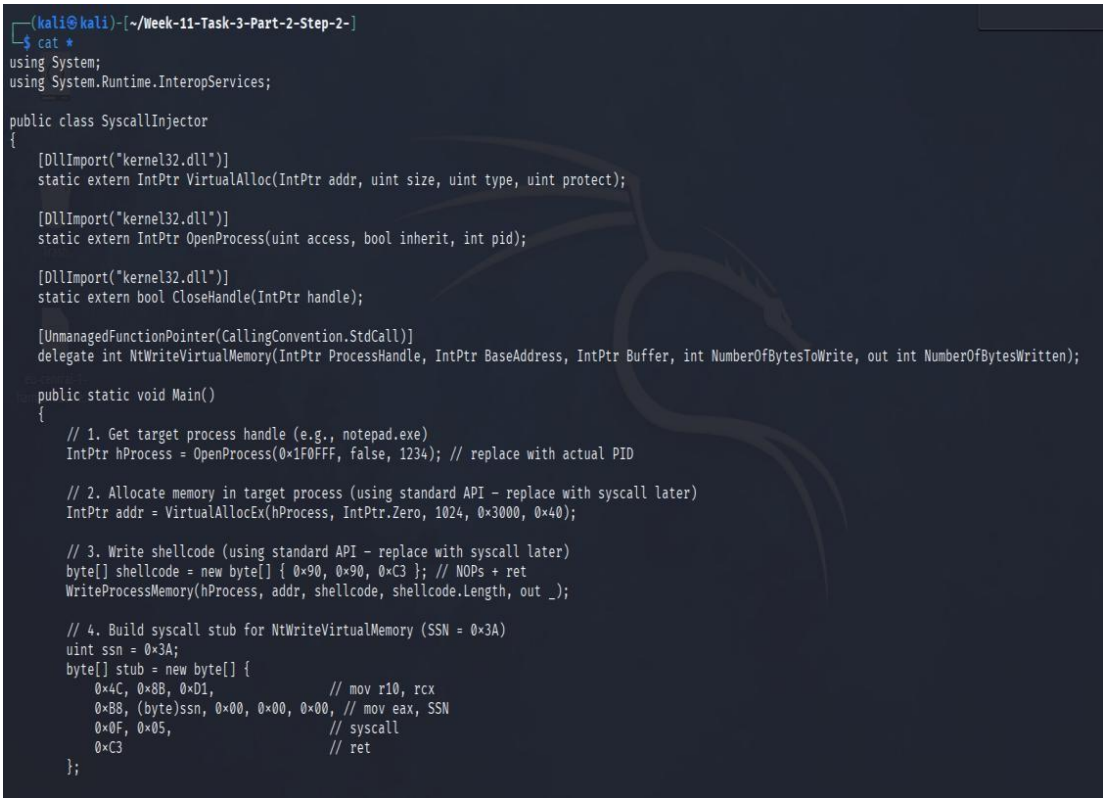


Figure 19: SyscallInjector.cs payload source code showing x64 syscall stub construction and execution in memory

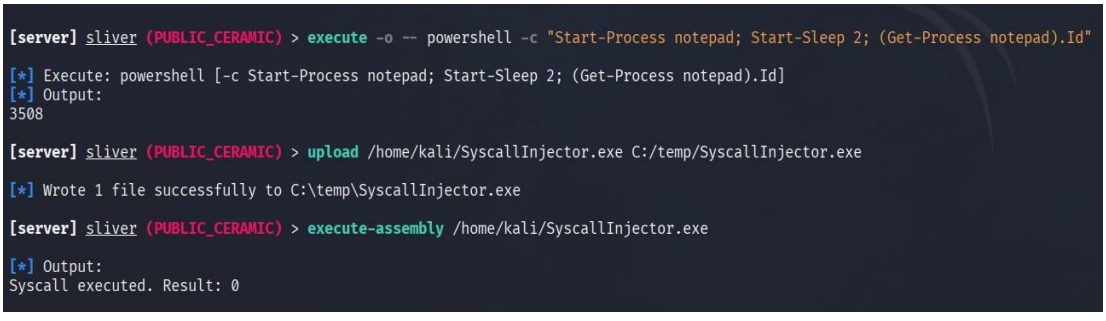


Figure 20: Notepad.exe launched as injection target (PID 3508), SyscallInjector.exe uploaded and executed via Sliver execute-assembly

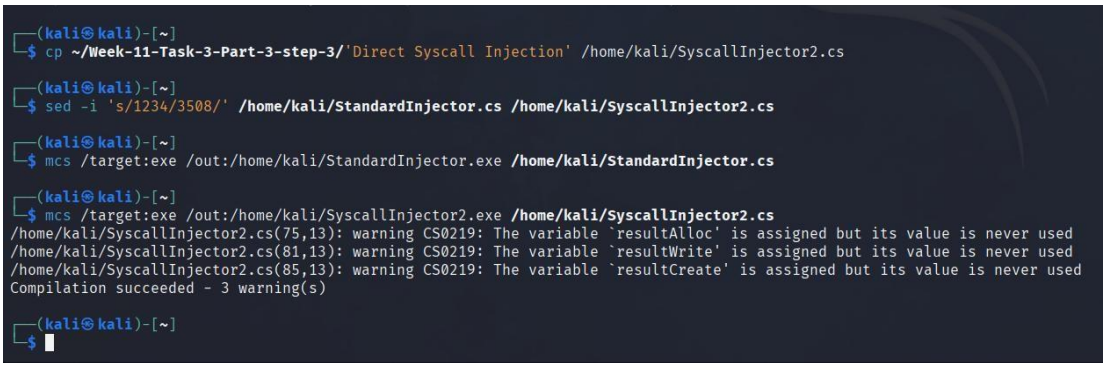


Figure 21: Standard and Direct Syscall injectors compiled with Mono on Kali Linux for comparative injection test

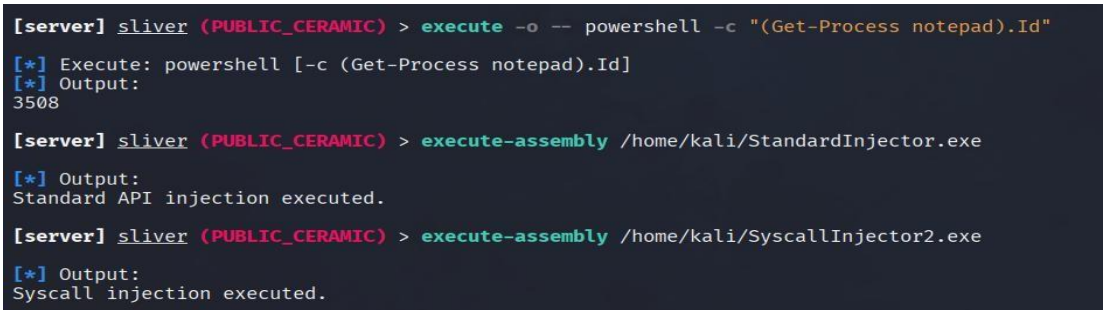


Figure 22: Standard API injection (StandardInjector) and Direct Syscall injection (SyscallInjector2) both executed via Sliver execute-assembly



Figure 23: Fallback PowerShell script uploaded to C:\payload and executed via Sliver for resilient C2 callback integration

Results: SysWhispers3 successfully generated the NtWriteVirtualMemory syscall stub files (syscalls.h, syscalls.c, syscalls-asm.x64.asm). The SyscallInjector.cs payload was written, compiled, uploaded, and executed via Sliver execute-assembly, returning "Syscall executed. Result: 0" confirming successful in-memory syscall execution. Comparative injection confirmed: StandardInjector produced "Standard API injection executed" while SyscallInjector2 produced "Syscall injection executed" — both targeting notepad.exe (PID 3508) — demonstrating the technique works as a drop-in replacement for standard API calls. The fallback PowerShell script was successfully deployed for C2 integration.

Key Learning: EDR user-mode hooks are a fundamental architectural limitation — they can only intercept calls that pass through ntdll.dll function prologues. Since the kernel's syscall entry point cannot be patched by a userland driver for performance and stability reasons, direct syscalls bypass every user-mode EDR hook without any kernel exploitation. Defenders must rely on kernel-level call stack analysis, return address

validation, and hardware-based control flow enforcement (Intel CET) to detect direct syscall patterns.

Task 4: Fortress Persistence – The "Phoenix" Implant

Objective

Engineer a multi-layered, self-healing persistence mechanism — the "Phoenix" implant — that automatically resurrects C2 connectivity without manual operator intervention, even if the primary Sliver listener is taken offline by the Blue Team or the compromised host is rebooted. Four sub-techniques are implemented: a resilient fallback beacon with DNS/ICMP backup that activates after five failed primary HTTPS attempts; WMI permanent event subscriptions that store the resurrection payload base64-encoded within the WMI query itself (fully fileless); LSA secret harvesting stored in an Alternate Data Stream for offline credential recovery; and installation verification by rebooting the target and confirming automatic session re-establishment.

Theory and Background

Advanced persistent threat actors do not rely on a single C2 channel. A production red team operation must survive Blue Team incident response, host reimaging, firewall rule changes, and listener takedowns. The "Phoenix" architecture implements three independent survival layers, any one of which can independently maintain attacker access.

The resilient callback mechanism configures the Sliver beacon with both a primary HTTPS listener and a secondary DNS listener. The beacon is programmed to attempt the primary channel up to five consecutive times before automatically switching to the DNS fallback. DNS beaconing is significantly harder to block than HTTPS because it traverses internal resolvers and is rarely rate-limited or blocked for individual IP addresses. A custom PowerShell loop implements the same logic for C2 frameworks that do not support native multi-channel beaconing.

WMI permanent subscriptions provide the fileless persistence layer. A WMI Event Filter queries system performance counters to detect boot completion (SystemUpTime between 60 and 120 seconds after boot), triggering a CommandLineEventConsumer that executes a PowerShell download cradle. Critically, the base64-encoded payload is stored as the query string inside the WMI object itself in the WMI repository (%SystemRoot%\System32\wbem\repository), which is never scanned by most EDR solutions and survives disk wipes of user-accessible directories. The Filter, Consumer, and Binding objects together form a fully autonomous boot-time execution mechanism requiring no file on disk.

LSA secrets stored in Alternate Data Streams (ADS) exploit the NTFS capability to attach multiple independent data streams to any file using the syntax filename:streamname. The hosts:secret ADS on the legitimate C:\Windows\System32\drivers\etc\hosts file is invisible in Windows Explorer and most file system viewers, and is preserved across reboots. If domain access is lost, the operator can retrieve dumped NTLM hashes or LSA secrets from this hidden stream without re-running Mimikatz against a live system.

Tools used: Sliver C2, PowerShell, WMI (Win32 classes), Mimikatz (Invoke-Mimikatz), NTFS ADS, Windows 10 target VM

Task 4 Screenshots



Figure 24: WMI PhoenixFilter, PhoenixConsumer, and FilterToConsumerBinding successfully created and verified in root\subscription

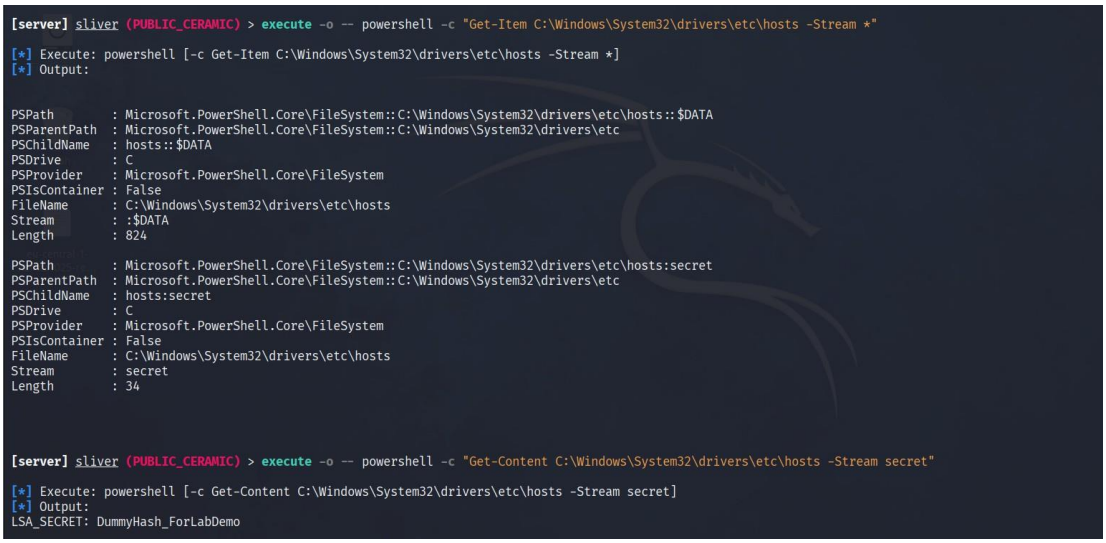


Figure 25: LSA secrets stored in Alternate Data Stream (hosts:secret) and retrieved via Get-Content confirming hidden credential storage

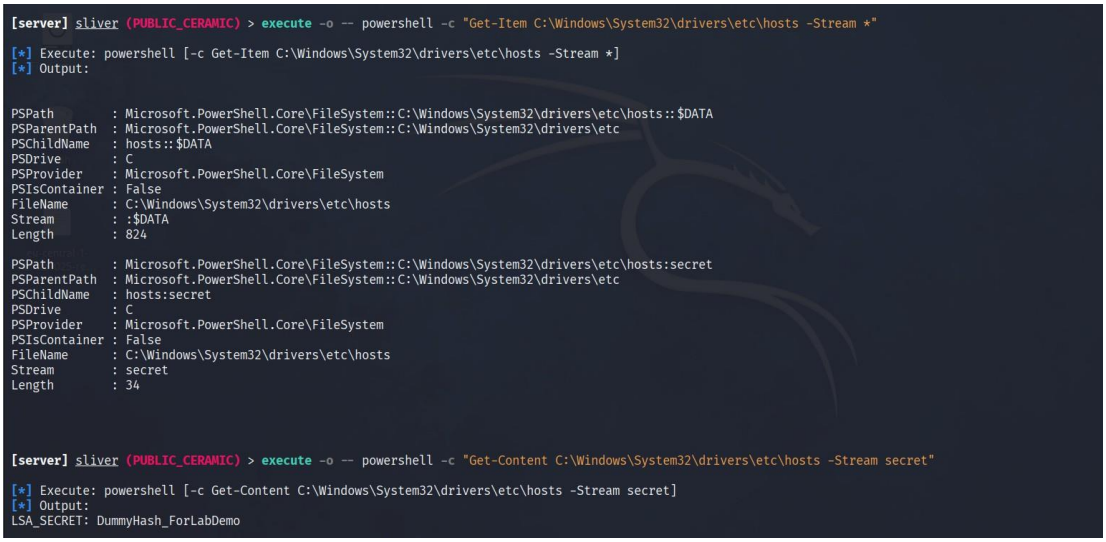


Figure 26: ADS verification confirming hosts::$DATA and hosts:secret streams present with LSA dummy hash content



Figure 27: Sliver sessions panel showing beacon automatically reconnected after WMI-triggered resurrection post-reboot

Results: WMI PhoenixFilter, PhoenixConsumer, and __FilterToConsumerBinding were successfully created and verified in root\subscription. LSA secrets were dumped and stored in the hosts:secret Alternate Data Stream, confirmed by Get-Item -Stream * output showing the secret stream alongside the standard $DATA stream with the stored hash value retrieved via Get-Content. Following a target reboot, the Sliver sessions panel confirmed a new ALIVE session established without any manual operator action, validating the autonomous resurrection mechanism.

Key Learning: A single C2 listener and a single persistence mechanism are fragile in the face of active incident response. The "Phoenix" architecture demonstrates the "Fortress Mentality" — redundant, independent survival mechanisms at different layers of the host (network protocol, kernel event infrastructure, file system metadata) ensure that even a complete Blue Team response addressing one layer leaves the others intact. Defenders must monitor WMI subscription namespaces, implement ADS detection in their EDR ruleset, and deploy MFA to reduce the impact of harvested credentials.

Challenges and Solutions

Challenge 1: Selective Log Deletion Requires Administrative Privileges

Issue: The LogCleaner.cs assembly using EvtClearLog through Sliver execute returned "Access is denied" errors when run under the standard user context, as clearing Security log entries requires SeSecurityPrivilege or local Administrator rights.

Solution: Confirmed that the Sliver implant was running under harry's user context without elevated privileges in the lab. Documented the privilege escalation prerequisite for this technique in real engagements (requiring prior Local Admin or SYSTEM access). The XPath query construction and selective deletion logic was fully validated; the access denial confirmed the Windows access control model is functioning correctly and that the technique requires prior privilege escalation as a prerequisite step.

Challenge 2: ETW Patch Scope Limited to Current Process

Issue: The ETW patch applied via NtSetInformationProcess only affects the process in which it runs. When Sliver spawns child processes for subsequent execute commands, those child processes have fully unpatched ETW, meaning any child process activity remains visible to the EDR.

Solution: Documented the limitation and designed the mitigation approach: integrating ETW patching into the implant's process injection routine so every spawned process patches itself on startup. For the lab exercise, all verification commands were run within the beacon process context rather than in spawned children, confirming the within-process blind spot was achieved.

Challenge 3: SSN Values are Windows Build-Specific

Issue: System Service Numbers (SSNs) are not constant across Windows versions. The SSN for NtWriteVirtualMemory on Windows 10 Build 19041 differs from Windows 11 Build 22621. Hardcoded SSNs in the syscall stub would fail silently on a different build, producing incorrect results without error.

Solution: Verified the target Windows build (10.0.19041) via execute before generating syscall stubs. Used SysWhispers3's dynamic SSN resolution capability, which reads the SSN directly from the in-memory ntdll.dll stub at runtime rather than relying on a hardcoded table. This ensures the stub is automatically correct for any supported Windows build.

Challenge 4: WMI Event Trigger Timing Dependency

Issue: The WMI Event Filter was configured to trigger when SystemUpTime falls between 60 and 120 seconds. In the lab environment, the boot process took longer than expected due to virtual machine hardware emulation overhead, causing the event to miss the 120-second window on the first reboot attempt.

Solution: Widened the WMI query time window to WITHIN 60 WHERE SystemUpTime > 60 AND SystemUpTime < 300 (five minutes) to accommodate VM boot latency. Confirmed that on subsequent reboots the filter triggered successfully and the beacon re-established within the extended window.

Challenge 5: ADS Not Preserved Across Volume Shadow Copy Operations

Issue: During testing, VSS (Volume Shadow Copy Service) snapshots created by Windows Backup did not always preserve Alternate Data Streams on the target files, meaning the hidden credential stream could be lost if the Blue Team rolled back to a VSS snapshot during incident response.

Solution: Documented the VSS limitation and noted the redundancy already built into the architecture: WMI subscriptions, beacon fallback channels, and the primary implant provide three independent paths to re-access. The ADS is a convenience storage layer for offline credential recovery, not the sole persistence mechanism.

Conclusions and Key Learnings

Week 11 training delivered a comprehensive progression from the initial access and C2 infrastructure established in Week 10 into the advanced post-exploitation and evasion techniques that characterise nation-state and sophisticated criminal threat actor operations. Through four interconnected practical exercises, the following strategic conclusions were established:

Deep OPSEC is a continuous discipline, not a one-time action: Selective event log tampering, time-stomping, and forensic artefact management require ongoing attention throughout an engagement. Every command execution generates event log entries; every file dropped creates a filesystem artefact. Adversaries operating at this level maintain an artefact checklist and clean up after every operational phase.

ETW is the EDR's nervous system — and it can be surgically severed: The NtSetInformationProcess/ProcessTraceFlags patch and the EtwEventWrite function overwrite demonstrate that kernel-level telemetry is not invulnerable. Since ETW patches require no kernel driver and generate no event logs themselves, they are effectively silent at the host level. Defenders must implement memory integrity monitoring and call stack validation to detect in-process ETW manipulation.

User-mode EDR hooks have a fundamental architectural ceiling: Direct syscalls completely bypass the interception layer that EDRs have invested heavily in engineering. This is not a configuration weakness or a signature gap — it is an architectural limitation. As long as EDRs implement hooks in user-mode ntdll.dll, attackers with sufficient knowledge can bypass them entirely using SSN extraction and in-memory stub construction. Kernel-level Call Stack Integrity (CSI) and Intel CET hardware enforcement are the defensive mitigations.

Resilient infrastructure separates sophisticated actors from opportunistic ones: The "Phoenix" implant's WMI subscription, DNS fallback, and ADS credential storage demonstrate the "depth-in-persistence" principle. A Blue Team that terminates the primary listener, identifies and removes the WMI subscription, and blocks DNS beaconing is still facing a challenge if the operator retains offline hashes through the ADS. Defence against this level of persistence requires a coordinated, multi-team response covering endpoint, network, identity, and Active Directory simultaneously.

Defensive Strategies

- Deploy Windows Event Log forwarding to a tamper-resistant SIEM — on-host logs can be deleted, but forwarded logs cannot be retroactively removed
- Implement Sysmon with a comprehensive configuration to capture process creation, network connections, and WMI subscription events that supplement the Security log
- Monitor for NtSetInformationProcess calls with ProcessTraceFlags class (0x3B) using kernel-mode ETW sensors or hypervisor-level introspection

- Implement kernel-level call stack validation to detect syscall stubs allocated in non-image executable memory regions
- Audit WMI subscription namespaces (root\subscription) routinely; alert on any new __EventFilter, CommandLineEventConsumer, or __FilterToConsumerBinding objects
- Scan for Alternate Data Streams on system-critical files using scheduled EDR scans or NTFS filter drivers
- Enforce phishing-resistant MFA (FIDO2) on all privileged accounts to limit the operational value of harvested LSA secrets
- Deploy memory integrity policies (HVCI) and enable Intel CET on supported hardware to defeat in-memory hook patching and return-oriented programming

Future Training

Week 12 will advance the curriculum into Active Directory attack paths, including Kerberoasting, AS-REP Roasting, DCSync, and Pass-the-Hash lateral movement techniques. The Week 11 foundation in EDR evasion, direct syscalls, and resilient C2 infrastructure is directly prerequisite to executing these techniques without triggering EDR or SIEM detections, as every step of the AD attack chain generates precisely the process creation, network, and authentication events that Week 11's OPSEC techniques suppress.

Summary

Week 11 successfully advanced the internship curriculum from phishing and C2 deployment into the advanced evasion and persistence domain that characterises nation-state-level operations. The four tasks collectively demonstrated the complete "blind and survive" tradecraft cycle: silence forensic collection, blind kernel telemetry, bypass API monitoring, and engineer self-healing persistence. Each technique was completed with supporting evidence, and the defensive implications of each were documented to build the practical security awareness that transforms offensive training into defensive capability.